Assignment 2: ReAct Agent Report

CE8014 AI 代理系統之設計與開發

2026-03-15

Section 1: Implementation Logic

1.1 Architecture Overview

本專案實作一個 text-based ReAct Agent，透過三階段流程回答複雜問題：

1. **ReAct Loop** –Thought -> Action -> Observation 迴圈，最多 5 步迭代
2. **Evidence Review Gate** –獨立 LLM 呼叫，驗證 draft answer 是否有充分證據支持
3. **Finalize Stage** –獨立 LLM 呼叫，進行 entity disambiguation 與最終答案合成

系統架構分為三個檔案：

- **tools.py** –工具層，提供 `search` 與 `calculate` 兩個工具
- **agent.py** –三階段 ReAct 核心，包含三組 System Prompt、Review Gate、Finalize Stage
- **main.py** –CLI 互動層，使用 prompt_toolkit 提供 slash commands

1.2 System Prompt Strategy

Agent 使用三組獨立的 System Prompt，各司其職：

SYSTEM_PROMPT（ReAct Loop）

主要 prompt，控制 Thought -> Action -> Observation 迴圈行為。設計策略：

1. 工具定義：明確列出可用工具及其呼叫格式(`search[query]`、`calculate[expression]`)，讓 LLM 知道自己的能力邊界。

2. 格式約束：嚴格定義每步輸出只能是「Thought + Action + PAUSE」或「Answer」二擇一，避免 LLM 自由發揮導致 parse 失敗。

3. Few-Shot Examples（3-shot）：嵌入三個完整的 ReAct 範例，分別展示不同能力：

- **Example 1: Task Decomposition**（日本與法國人口比例）–展示問題拆解：先搜日本人口，再搜法國人口，最後計算比例。同時展示工具串接（搜尋結果送入 calculate）與格式遵守。
- **Example 2: Reflection After Poor Results**（Instagram 早期技術棧）–展示搜尋結果不理想時如何反思並調整查詢。第一次搜尋只找到「目前」的技術棧，模型主動識別問題並改用更精確的查詢找到「第一版」的資訊。
- **Example 3: Ambiguity Handling**（Notion CEO）–展示同名實體消歧義。搜尋結果顯示多個名為 “Notion” 的公司（生產力工具 vs AI 研究公司），模型嘗試區分後正確報告歧義。

Few-Shot 範例的有效性在於：Example 1 與 Task 1（日本 vs 台灣人口）結構高度相似，LLM 能直接遷移 decomposition pattern；Example 2 為 Task 3 的 Reflection 能力提供示範；Example 3 展示了 entity disambiguation 的正確處理方式。同時，範例中的 Observation 格式與實際搜尋結果格式一致，減少格式適應成本。

4. Source-First Observation：搜尋結果格式經過設計，每筆結果包含 Domain / URL / Snippet 三欄，讓 LLM 能根據來源可信度判斷資訊品質。Tavily 的 AI summary 被標記為 `unverified hint`，引導 LLM 優先信任原始 snippet。

5. Epistemic Guardrails：System Prompt 中嵌入多條認知規範：

- **Grounded Assertion**：「Only assert a fact if you can point to a specific snippet」–防止模型空口宣稱事實
- **Targeted Verification**：允許使用已找到的線索（人名、URL）進行驗證式搜尋，屬合理調查
- **Anti-Confirmation Bias**：不假設候選答案正確，需確認來源是否指向問題所述的同一實體
- **Entity Disambiguation**：「list the distinct entities you see before continuing」–強制模型在同名實體間做區分
- **Inconclusive Fallback**：允許模型在證據不足時輸出「evidence is inconclusive」，而非強迫猜測

REVIEW_SYSTEM_PROMPT（Evidence Review Gate）

獨立的 LLM 呼叫，接收 question、draft answer、ReAct trace，輸出結構化 JSON：

```
{
  "decision": "accept | revise",
  "reason": "...",
  "next_query": "...",
  "selected_person": "...",
  "supporting_quotes": ["..."],
  "descriptor_match": true | false,
  "ambiguity_detected": true | false
}
```

核心規則：

- `descriptor_match` 只有在 evidence 明確匹配問題中的 product/domain 描述時才為 true
- `ambiguity_detected` 在多個同名實體出現時為 true
- `decision` 必須在以下任一條件成立時為 `revise`：descriptor_match 為 false、ambiguity_detected 為 true、selected_person 不被 supporting_quotes 直接支持
- `next_query` 在 `decision` 為 `revise` 時必須非空，應直接針對證據缺口提出具體查詢

FINALIZE_SYSTEM_PROMPT（Final Synthesis）

最終合成階段，進行 entity disambiguation 與答案生成，輸出 JSON：

```
{
  "answer_type": "resolved | inconclusive",
  "selected_person": "...",
  "selected_entity": "...",
  "supporting_quotes": ["..."],
  "conflicting_entities": ["..."],
  "final_answer": "..."
}
```

核心規則：

- 不得 invent 不在 observations 中的事實
- supporting_quotes 必須從 observation snippets 中逐字引用
- 若 selected_person 沒有被 supporting_quotes 直接支持，answer_type 必須為 inconclusive
- **Entity Match 驗證**：若問題指定了特定 product type 或 industry（如「AI search」），確認 observations 中的實體確實在該領域營運；若為同名不同實體，設 answer_type 為 inconclusive 並列出 conflicting entities

完整 SYSTEM_PROMPT 如下：

```
You are a research assistant that answers questions using a
Thought -> Action -> Observation loop (ReAct pattern).

You have access to the following tools:
- search[query]: Search the web for current information.
  Use specific, targeted queries.
- calculate[expression]: Evaluate a math expression
  (supports +, -, *, /, **).

On each step you MUST output exactly ONE of:
1. A Thought line followed by an Action line, then PAUSE
   (wait for observation)
2. An Answer line (when you have enough information)

Format rules:
- Thought: <your reasoning>
- Action: <toolname>[<argument>]
- PAUSE
- Answer: <your final answer>

IMPORTANT:
- When you have enough information to answer, you MUST start
  your response with "Answer:" on its own line. Do NOT output
  your conclusion without the "Answer:" prefix.
- After writing an Action line, you MUST write PAUSE and stop.
  Do NOT write Observation yourself.
- The system will provide the Observation after your PAUSE.
- Treat search summaries as hints, not as verified facts.
  Prefer source titles, domains, and snippets.
- Only assert a fact if you can point to a specific snippet
  from the Observations that states it.
- If a search returns poor, conflicting, or off-topic results,
  reflect on why and try a DIFFERENT query angle.
- When different search results appear to refer to different
  entities with the same name, list the distinct entities
  and refine your query with neutral qualifiers (industry,
```

```
  product type, headquarters, or domain) before answering.
- AVOID CONFIRMATION BIAS: Do not assume a candidate answer
  is correct just because one source mentions it.
- If ambiguity remains after a reasonable retry, give an
  Answer that says the evidence is inconclusive.
- Break complex questions into sub-questions and solve them
  step by step.

=== EXAMPLE 1: Task Decomposition ===

Question: What is the population ratio of Japan to France?
Thought: I need to find the populations of both Japan and
France. Let me start with Japan.
Action: search[Japan population 2024]
PAUSE

Observation:
[1] RESULT
Title: Japan Demographics
Domain: example.com
Snippet: Japan's population was estimated at 123.3 million
in 2024...

Thought: Japan's population is about 123.3 million.
Now I need France's population.
Action: search[France population 2024]
PAUSE

[...observation...]

Thought: Now I can calculate the ratio.
Action: calculate[123.3 / 68.4]
PAUSE

Observation: 1.8026315789473686

Thought: The ratio is approximately 1.80.
Answer: The population ratio of Japan to France is
approximately 1.80:1.

=== EXAMPLE 2: Reflection After Poor Results ===

Question: What programming language was used to build the
first version of Instagram?
Thought: I need to find information about Instagram's tech
```

```
stack in its early days.
Action: search[Instagram programming language]
PAUSE

Observation:
[1] RESULT
Title: Instagram Engineering Blog
Snippet: Instagram now uses Python, Django, and various
other technologies...

Thought: The search results show Instagram's current tech
stack, but the question asks about the FIRST version.
Let me try a more specific query.
Action: search[Instagram first version original tech stack 2010]
PAUSE

[...observation showing first version was Python/Django...]

Thought: Now I have the specific information about the
first version.
Answer: The first version of Instagram, launched in 2010,
was built using Python with the Django framework.

=== EXAMPLE 3: Ambiguity Handling ===

Question: Who is the CEO of Notion?
Thought: I need to find Notion's CEO.
Action: search[Notion CEO]
PAUSE

Observation:
[1] RESULT
Title: Notion - The all-in-one workspace
Domain: notion.so
Snippet: Notion is a productivity and note-taking app.
Founded by Ivan Zhao and Simon Last.

[2] RESULT
Title: Notion AI - Research Lab
Domain: notion.ai
Snippet: Notion AI is an artificial intelligence research
company based in San Francisco...

Thought: I found two different companies named "Notion":
notion.so (productivity workspace) and notion.ai (AI
```

```
research company). I need to clarify and search more
specifically.
Action: search[Notion workspace productivity app notion.so CEO]
PAUSE

[...observation confirming Ivan Zhao for notion.so...]

Thought: I have identified two distinct entities with the
name "Notion". Since the question doesn't specify, I should
report this ambiguity.
Answer: There are multiple companies named "Notion". If you
mean Notion (notion.so), the productivity workspace, the CEO
is Ivan Zhao. If you mean Notion AI (notion.ai), an AI
research company, the leadership differs. Please specify
which company you are referring to.

=== END EXAMPLES ===
```

Python 如何將 Observation 回饋至 LLM context window

ReAct Loop 以 messages list 維護完整對話歷史。每次工具呼叫後，Observation 作為 user role 訊息注入：

```python
# LLM 輸出的 Action 記為 assistant message
messages.append({"role": "assistant", "content": raw + "\nPAUSE"})

# 工具執行結果作為 user message 注入
tool_func = TOOL_REGISTRY[tool_name]
observation = tool_func(tool_arg)
messages.append({"role": "user", "content": f"Observation: {observation}"})
```

這確保 LLM 在每個 iteration 都能看到完整的 Thought-Action-Observation 歷史。Review Gate 和 Finalize Stage 各自使用獨立的 messages（只包含自己的 system prompt + 從主 loop 提取的 trace），不會污染主 loop 的 context。

1.3 Loop Mechanism

三階段流程

```python
messages = [system_prompt, user_query]
draft_answer = None
review_budget = 2  # review gate 最多觸發兩次 retry search

for i in 1..MAX_ITERATIONS(5):
    response = LLM(messages, stop=["PAUSE", "Observation:"])
```

```
        if "Answer:" in response:
            draft_answer = extract_answer(response)

            # --- Stage 2: Evidence Review Gate ---
            review = _review_draft(question, messages, draft_answer)

            if review.decision == "accept":
                break  # proceed to finalize

            if i < MAX and review_budget > 0:
                review_budget -= 1
                run_search(review.next_query)  # one targeted retry
                continue

            break  # budget exhausted, go to finalize anyway

        if "Action:" in response:
            result = dispatch_tool(action, arg)
            messages.append(observation)
            continue

        messages.append(nudge_message)

    # --- Stage 3: Finalize ---
    final_answer = _finalize_answer(question, messages, draft_answer)
    return final_answer
```

Stage 2: Evidence Review Gate (`_review_draft`)

Review Gate 分為兩層檢查：

第一層：Structural Guard（結構化前置檢查）–在呼叫 LLM 之前，Python 端計算目前 messages 中的 observation 數量。若 `obs_count < 2`，直接返回 `revise`，跳過 LLM review。這是防止模型在只有一筆搜尋結果時就過早收斂的安全網。當 structural guard 觸發時，follow-up query 由獨立 LLM 呼叫（`_suggest_followup_query`）生成，使用中性 descriptor 避免 confirmation bias。

第二層：LLM 語意審查–當 observation >= 2 時，進入完整的 LLM review。使用 `REVIEW_SYSTEM_PROMPT`，接收 question、draft answer、ReAct trace，輸出結構化 JSON。Python 端只做基本結構驗證（確認 `decision` 為 `accept` 或 `revise`），語意判斷委託給 LLM。

`review_budget = 2` 限制 review gate 最多觸發兩次 retry search，為 entity disambiguation 提供足夠的搜尋空間。若 review 判定 `accept`，agent 直接回傳 draft answer，跳過 finalize 階段以避免不必要的 API 呼叫和答案漂移。

Stage 3: Finalize (`_finalize_answer`)

Finalize 同樣是獨立 LLM 呼叫，使用 `FINALIZE_SYSTEM_PROMPT`。Python 端信任 LLM 的結構化輸出：若 `answer_type` 為 `inconclusive` 則使用 inconclusive 答案（含 `conflicting_entities` 資訊），否則直接採用 `final_answer`。

Stop Sequence

API 呼叫時設定 `stop=["PAUSE", "Observation:"]`，這是防止 LLM hallucinate Observation 的關鍵機制：

- **PAUSE**：主停止點。LLM 寫完 Action 後應輸出 PAUSE，API 在此截斷回應。
- **Observation:**：Fallback 停止點。若 LLM 試圖自行編造 Observation，API 會在此截斷。

被截斷後，程式端補回 PAUSE 到 messages history，確保 LLM 在後續 iteration 看到一致的格式。

Parse Logic

使用 regex 解析 LLM 輸出：

- `Answer: (.+)` -> 進入 Review Gate
- `Action: (\w+)\[(.+?)\]` -> 呼叫對應工具，將結果作為 Observation 注入
- 都沒匹配 -> 送 nudge 訊息提醒 LLM 使用正確格式，不直接中斷

Nudge Mechanism

當偵測到「既無 Action 也無 Answer」時，不中斷迴圈，而是注入提示訊息讓 LLM 在下一個 iteration 用正確格式補上回答。

1.4 Tool Design

search

使用 Tavily Python SDK，設定 `search_depth="advanced"`、`max_results=5`、`include_answer=True`。回傳格式為 source-first 結構：

```
[1] Title
    Domain: example.com
    URL: https://...
    Snippet: ...

Search summary (unverified hint; prefer the source snippets
above if they conflict): ...
```

每筆結果包含 Domain / URL / Snippet，讓 LLM 能根據來源判斷可信度。Tavily AI summary 標記為 unverified hint，避免模型過度信任合成摘要。

Snippet 長度上限 500 字元，超過部分截斷並加 `...`，防止單一搜尋結果佔用過多 context window。錯誤一律轉為字串回傳，讓 Agent 自行 reflect 而非 crash。

calculate

使用 Python `ast` 模組安全解析數學運算式，只允許 `+, −, *, /, **` 五種運算符。透過遞迴走訪 AST 節點實現，完全阻擋 `eval` injection（如 `__import__("os")`）。

Section 2: Benchmark Traces

以下為三個 Task 的完整 console trace 與分析。所有 Task 使用同一個 `ReActAgent` 類別，每次 `run()` 呼叫重置 messages（三個 Task 為獨立問題，不需跨問題記憶）。

Task 1: Population Fraction

Question: *What fraction of Japan's population is Taiwan's population as of 2025?*

Console Trace

```
--- Step 1/5 ---
  Thought: I need to find the populations of both Japan and
  Taiwan for the year 2025. I'll start by searching for
  Japan's population in 2025.
  Action: search[Japan population 2025]
  Observation:
    [1] Japan Population (1950-2025) - Macrotrends
        Domain: macrotrends.net
        Snippet: Total current population for Japan in 2025
        was 121,960,408, a 0.55% decline from 2024.
    [2] Demographics of Japan - Wikipedia
        Domain: en.wikipedia.org
    ... (19 more lines)

--- Step 2/5 ---
  Thought: Japan's population in 2025 is approximately
  123.1 million. Now, I need to find Taiwan's population
  for the same year.
  Action: search[Taiwan population 2025]
  Observation:
    [1] Taiwan Population - Trading Economics
        Domain: tradingeconomics.com
        Snippet: The total population in Taiwan was estimated
        at 23.3 million people in 2025.
    [2] Taiwan Population (2026) - Worldometer
        Domain: worldometers.info
    ... (19 more lines)

--- Step 3/5 ---
  Thought: Taiwan's population in 2025 is approximately
  23.3 million. Now I can calculate the fraction of Japan's
  population that Taiwan's population represents.
  Action: calculate[23.3 / 123.1]
  Observation: 0.1892770105605199

--- Step 4/5 ---
```

```
Thought: Draft answer passed evidence review.

Answer: The fraction of Japan's population that is Taiwan's
population as of 2025 is approximately 0.189, or about 18.9%.
```

Analysis

- **Decomposition**：Agent 正確將問題拆解為三個子任務–搜尋日本人口、搜尋台灣人口、計算比例。
- **Tool Chaining**：搜尋結果成功傳遞給 calculate 工具，展現跨工具協作能力。
- **Few-Shot 效果**：Task 1 的結構與 System Prompt 中 Example 1（日本 vs 法國）高度相似，LLM 完美複製了這個 decomposition pattern。
- **Review Gate**：draft answer 直接通過 review（observation >= 2、無歧義、data grounded）。
- **步數**：4 步完成（2 次搜尋 + 1 次計算 + 1 次 answer），效率合理。

Task 2: Display Specs Comparison

Question: *Compare the main display specs of iPhone 15 and Samsung S24.*

Console Trace

```
--- Step 1/5 ---
  Thought: I need to find the display specifications for both
  the iPhone 15 and the Samsung S24. I'll start with iPhone 15.
  Action: search[iPhone 15 display specifications]
  Observation:
    [1] iPhone 15 Pro - Tech Specs - Apple Support
        Domain: support.apple.com
        Snippet: Super Retina XDR display 6.1-inch OLED
        2556-by-1179-pixel resolution at 460 ppi
        Dynamic Island, ProMotion 120Hz, HDR, 2000 nits
    [2] iPhone 15 - Wikipedia
        Domain: en.wikipedia.org
    ... (19 more lines)

--- Step 2/5 ---
  Thought: I have gathered the display specifications for
  the iPhone 15. Now, I need to find the display specs for
  Samsung S24.
  Action: search[Samsung S24 display specifications]
  Observation:
    [1] Samsung Galaxy S24 Specifications
        Domain: cuit.columbia.edu
        Snippet: 6.2" FHD+ Dynamic AMOLED 2X, 120Hz
        Gorilla Glass Victus 2, IP68
    [2] Samsung Galaxy S24
        Domain: en.wikipedia.org
    ... (19 more lines)

--- Step 3/5 ---
  Thought: I have gathered the display specifications for
  both the iPhone 15 and the Samsung S24. Now I can summarize
  the main display specs for comparison.
  [!] No Action or Answer detected, nudging LLM...

--- Step 4/5 ---
  Thought: Draft answer passed evidence review.

  Answer: The main display specifications are as follows:
  iPhone 15: Super Retina XDR OLED, 6.1", 2556 x 1179,
    60 Hz, 2000 nits, HDR, True Tone, Dynamic Island
  Samsung S24: Dynamic AMOLED 2X, 6.2", 1080 x 2340,
    120 Hz, 2600 nits, HDR10+, Gorilla Glass Victus 2
```

```
The iPhone 15 has a higher resolution and pixel density,
while the Samsung S24 has a higher refresh rate and
peak brightness.
```

Analysis

- **Search Strategy**：Agent 分別搜尋兩支手機的規格，共兩次搜尋取得完整資訊。
- **具體規格差異**：成功擷取到 60Hz vs 120Hz refresh rate、resolution 差異（2556x1179 vs 1080x2340）、亮度差異（2000 vs 2600 nits）等關鍵規格。
- **格式遵從問題**：Step 3 中 LLM 直接輸出比較內容但遺漏 `Answer:` 前綴，觸發 nudge 機制。這是 gpt-4o-mini 在 instruction following 上的常見問題–當資訊充足時，模型傾向直接輸出答案而忽略格式約束。Nudge 機制在此展現了其必要性。
- **步數**：4 步完成（2 次搜尋 + 1 次 nudge + 1 次 answer）。

Task 3: Morphic CEO (Resilience & Reflection Test)

Question: *Who is the CEO of the startup ‘Morphic’ AI search?*

此 Task 是最具挑戰性的測試，因為「Morphic」是多個不同公司或專案的名稱：

- **morphic.sh** –開源 AI search engine（問題所指的目標），由 Yoshiki Miura 開發維護（GitHub: miurla），屬個人開源專案
- **morphic.com** –AI 影片/動畫平台（Jaynti Kanani, Polygon 前共同創辦人）
- **getmorphic.com** –AI 網站建置平台（Weston Baker, Founder）
- **Morphic Therapeutic** –生技公司（Praveen Tipirneni, CEO）
- **Morphic Medical** –醫療器材公司（Mike Gutteridge, CEO）

搜尋引擎對「Morphic CEO」的回傳結果會混合這些不同實體，構成 entity disambiguation 挑戰。

Console Trace (gpt-4o-mini)

```
--- Step 1/5 ---
  Thought: I need to find information about the CEO of the
  startup 'Morphic' that focuses on AI search.
  Action: search[Morphic AI search CEO]
  Observation:
    [1] Morphic - 2026 Company Profile & Team - Tracxn
        Domain: tracxn.com
        Snippet: Jaynti Kanani is the current CEO of Morphic.
    [2] Morphic Management Team - CB Insights
        Domain: cbinsights.com
    ... (19 more lines)

--- Step 2/5 ---
  Thought: There are conflicting sources regarding the CEO
  of Morphic. One source states Jaynti Kanani, while another
  claims Weston Baker. I need to clarify.
  Action: search[Morphic AI search company CEO clarification]
  Observation:
    [1] Morphic - 2026 Company Profile & Team - Tracxn
        Domain: tracxn.com
        Snippet: Jaynti Kanani is the current CEO of Morphic.
    [2] Morphic Management Team - CB Insights
        Domain: cbinsights.com
    ... (19 more lines)

--- Step 3/5 ---
  Thought: The search results continue to show conflicting
  information. I will check the official website of Morphic.
  Action: search[Morphic official website]
  Observation:
    [1] About - Morphic
```

```
        Domain: morphic.com
        Snippet: Morphic is a platform designed to empower
        creators, filmmakers, and animators in producing
        high-quality stories using AI.
    [2] Morphic makes computers easier to use
        Domain: morphic.org
    ... (19 more lines)

--- Step 4/5 ---
  Thought: The official website does not provide specific CEO
  information. I will report the ambiguity in my answer.
  [!] Evidence review requested more search: The draft answer
      mentions conflicting reports but does not clearly state
      which source is more reliable.
  Thought: Running one targeted disambiguation search.
  Action: search[Clarify the current CEO of Morphic AI search]
  Observation:
    [1] Who is the CEO of Morphic Therapeutic?
        Domain: clay.com
        Snippet: Praveen Tipirneni is the CEO of Morphic
        Therapeutic, a biotech company.
    [2] Morphic to Produce AI Anime Series - Variety
        Domain: variety.com
    ... (19 more lines)

--- Step 5/5 ---
  [!] Evidence review found gaps; review budget exhausted.
      Moving to final synthesis.

--- Finalize ---
  Thought: Synthesizing evidence and resolving ambiguity.

  Answer: The current CEO of Morphic AI is Jaynti Kanani,
  as stated in multiple sources. However, there is conflicting
  information regarding the leadership of Morphic, as another
  source lists Weston Baker as the CEO. This ambiguity suggests
  that there may be multiple entities named Morphic.
```

MiniMax M2.5 對照實驗

為對照不同模型在相同架構上的表現，以相同程式碼切換至 MiniMax M2.5 進行 Task 3 測試：

```
--- Step 1/5 ---
  Thought: I need to find information about a startup called
  "Morphic" related to AI search and who its CEO is.
  Action: search[Morphic AI search startup CEO]
  Observation:
```

```
    [1] Eric Vardon is Co-Founder and CEO of Morphio.ai
        Domain: youtube.com
    [2] Morphic - 2026 Company Profile & Team - Tracxn
        Domain: tracxn.com
    ... (19 more lines)

--- Step 2/5 ---
  Thought: The results show multiple companies named "Morphic".
  I need to search more specifically for the AI search startup.
  Action: search["Morphic" AI search engine startup]
  Observation:
    [1] $1 Million up for grabs for emerging creators.
        Domain: instagram.com
    [2] Morphic launches AI anime series - LinkedIn
        Domain: linkedin.com
    ... (19 more lines)

--- Step 3/5 ---
  Thought: I found Morphic is an AI-powered search engine.
  Now I need to find the specific CEO/founder.
  Action: search[Morphic AI search engine founder CEO]
  Observation:
    [1] Morphic - 2026 Company Profile - Tracxn
        Domain: tracxn.com
        Snippet: Jaynti Kanani is the current CEO of Morphic.
    [2] Eric Vardon is Co-Founder and CEO of Morphio.ai
        Domain: youtube.com
    ... (19 more lines)

--- Step 4/5 ---
  Thought: I found a reference to "JD Kanani, Founder & CEO"
  on morphic.com. Let me verify this is the correct Morphic.
  Action: search["JD Kanani" Morphic founder CEO]
  Observation:
    [1] Jaynti Kanani - Founder & CEO at Morphic - The Org
        Domain: theorg.com
        Snippet: Jaynti Kanani holds the position of Founder
        & CEO at both Mozak and Morphic.
    [2] Jaynti Kanani Email - RocketReach
        Domain: rocketreach.co
    ... (19 more lines)

--- Step 5/5 ---
  Thought: I need to verify that Jaynti Kanani is indeed
  the CEO of the Morphic that operates the AI search engine.
  Action: search[morphic.sh founder CEO Jaynti Kanani]
  Observation:
    [1] Jaynti Kanani - Founder & CEO at Morphic - The Org
```

```
        Domain: theorg.com
    [2] Jaynti Kanani
        Domain: contra.com
    ... (19 more lines)

--- Finalize ---
  Thought: Synthesizing evidence and resolving ambiguity.

  Answer: The search results reveal multiple companies named
  "Morphic" operating in different domains: morphic.sh
  (AI search engine), morphic.com (AI storytelling/animation),
  and others. Without explicit verification linking Jaynti
  Kanani directly to the AI search engine division, the
  answer remains inconclusive.
```

Analysis

gpt-4o-mini 的行為：

- Step 1-2 正確識別了搜尋結果中的衝突（Jaynti Kanani vs Weston Baker），展現初步 reflection
- Step 3 搜尋 `morphic.com/about` 發現是「creators, filmmakers, and animators」平台，但模型沒有區分「AI animation」vs「AI search」
- **Entity Mismatch 未被偵測**：模型錯誤地認定 Jaynti Kanani 為「Morphic AI search」的 CEO，未區分 morphic.com（AI 影片/動畫平台）與 morphic.sh（AI 搜尋引擎）
- Review Gate 在 Step 4 偵測到證據缺口，觸發一次額外搜尋
- Step 5 review budget 用完，進入 Finalize
- **Finalize 未能修正錯誤**：最終輸出錯誤答案，指出 Jaynti Kanani 為 CEO 但承認有歧義

MiniMax M2.5 的行為：

- Step 1-2 識別出搜尋結果包含不同公司，主動縮窄搜尋範圍
- Step 3-4 持續搜尋，嘗試確認 Jaynti Kanani 是否為 AI search engine 的 CEO
- Step 5 搜尋 `morphic.sh founder CEO` 但結果仍被其他同名實體干擾
- Finalize 正確輸出 inconclusive，列出多個不同 Morphic 實體

Reflection 路徑對比：

階段	**gpt-4o-mini**	**MiniMax M2.5**
Entity 識別	識別衝突但未深入區分	逐步區分不同實體
搜尋策略	4 次搜尋 + 1 次 review 觸發的搜尋	5 次搜尋
Entity Mismatch	未偵測（animation vs AI search）	正確識別歧義
Review Gate	Step 4 觸發額外搜尋	未進入（5 步用完）
最終結果	Incorrect（有歧義但給答案）	Inconclusive（正確的保守輸出）

Section 3: Model Capability Analysis

跨模型對照實驗

使用相同 agent 程式碼（同一個 `ReActAgent` class、同一組 system prompt），僅透過 `.env` 切換 `MODEL` 環境變數，對三個 Task 進行跨模型測試：

Model	Task 1	Task 2	Task 3
gpt-4o-mini	正確（18.9%）	正確（需 nudge）	錯誤（entity mismatch）
MiniMax M2.5	正確（18.9%）	正確（一次通過）	Inconclusive（正確消歧）

發現

1. 格式遵守能力差異：

gpt-4o-mini 在 Task 2（Step 3）中觸發了 nudge –模型在資訊充足時傾向直接輸出比較結果而忽略 `Answer:` 前綴。Task 3 中因 SYSTEM_PROMPT 加入了「you MUST start your response with Answer:」的強制指示，nudge 未再發生。MiniMax M2.5 在相同架構下從未出現此問題。

2. Entity Disambiguation 能力差距：

Task 3 是區分模型推理能力的關鍵測試。核心差異在於 **product type mismatch 偵測**：

gpt-4o-mini：搜尋到 Variety 文章明確描述 Jaynti Kanani「launched Morphic in 2024」做 AI anime 後，直接認定其為「Morphic AI search」的 CEO。模型未能區分「AI anime production」（morphic.com）與「AI search engine」（morphic.sh）。即使使用了 targeted verification query（嵌入候選人名搜尋），模型在 4 次搜尋後仍未能正確區分 product type –這是語意推理層面的能力不足，而非搜尋策略問題。

MiniMax M2.5：在 Step 3 中明確區分了 Weston Baker（getmorphic.com 網頁設計），Step 4-5 逐步縮窄至 morphic.sh 但搜尋結果被其他同名實體干擾，在無法確認 morphic.sh 具體開發者的情況下正確輸出 inconclusive。

3. Structural Guard 的效果與局限：

`_review_draft` 中的 observation 數量門檻（`obs_count < 2`）有效防止了 gpt-4o-mini 在第一次搜尋後就過早收斂。但結構化檢查只能確保「搜了足夠多次」，無法確保「搜到正確的東西」–當模型在語意層面無法區分同名不同產品的實體時，再多的搜尋也不會改善結果。這驗證了 structural guard 是必要的安全網，但不是萬能的解決方案。

4. Inconclusive 是合理的安全輸出：

MiniMax M2.5 在 Task 3 的結果是 inconclusive。morphic.sh 實際上是 Yoshiki Miura 的個人開源專案，搜尋結果中出現的 Jaynti Kanani（morphic.com, AI 影片平台）和 Weston Baker（getmorphic.com, 網站建置）都是不同的 Morphic。在此條件下輸出 inconclusive 並列出衝突實體，是合理的判斷。

5. 架構設計原則–分層防禦：

Python orchestration 層設置最小限度的結構化檢查（如 observation 數量門檻）作為安全網，核心語意判斷委託 LLM structured output 處理。過多的 Python-side 語意驗證容易產生 false negative，過少則無法防止模型過早收斂。

Summary

	Task 1	Task 2	Task 3 (4o-mini)	Task 3 (MiniMax)
Steps Used	4/5	4/5	5/5	5/5
Search Calls	2	2	4 + 1 (review)	5
Calculate Calls	1	0	0	0
Decomposition	Yes	Yes	N/A	N/A
Reflection	No (unnecessary)	No (unnecessary)	Yes (partial)	Yes (full)
Nudge Triggered	No	Yes (Step 3)	No	No
Review Gate	Accept	Accept	Revise + Retry	N/A (5 步用完)
Result	Correct	Correct	Incorrect	Inconclusive

Task 1、2 在 4 步內主動完成；Task 3 兩個模型都用滿 5 步上限，是被迫結束而非主動收斂–這反映了 entity disambiguation 問題的搜尋複雜度超出 5 步預算。整體而言，三個 Task 展現了 ReAct pattern 的核心能力與局限：

1. **Task Decomposition**：複雜問題（Task 1, 2）被自動拆解為多個子問題依序處理。3-shot Few-Shot 範例有效引導模型採取正確的 decomposition 策略。
2. **Tool Selection**：Agent 根據子問題性質選擇適當工具（搜尋 vs 計算）。
3. **Resilience & Reflection**：Task 3 展示了 self-correction 流程的完整光譜–gpt-4o-mini 在 Thought 中識別了搜尋結果的衝突（reflection），但在語意推理層面未能區分 AI animation（morphic.com）與 AI search（morphic.sh），最終給出錯誤答案；MiniMax M2.5 則透過更精確的 entity disambiguation，正確判定證據不足並輸出 inconclusive。
4. **Model-Agnostic Architecture**：同一套 agent 程式碼在不同模型上展現不同水平的表現，驗證了架構設計的可遷移性。Structural guard 提供了最低安全保障，但最終結果品質仍取決於模型本身的語意推理能力。
5. **Few-Shot Design**：三個 Few-Shot 範例分別展示 Task Decomposition、Reflection、Ambiguity Handling 能力。Example 2 與 Example 3 為 Task 3 的 Reflection 與 Entity Disambiguation 提供行為示範，但 gpt-4o-mini 在 product type mismatch 偵測上仍受限於模型本身的語意推理能力。